Lab Assignment 9.5

1. String Utilities Function:

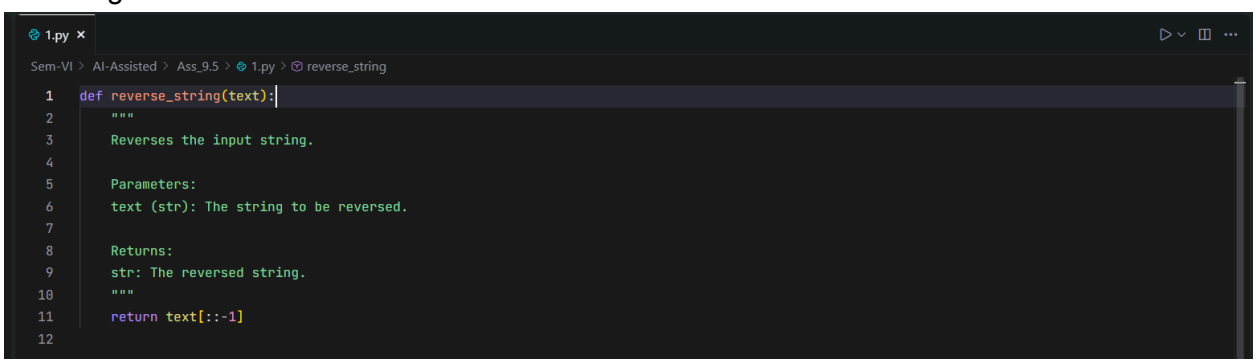
   Consider the following Python function:

```
def reverse_string(text):
return text[::-1]
```

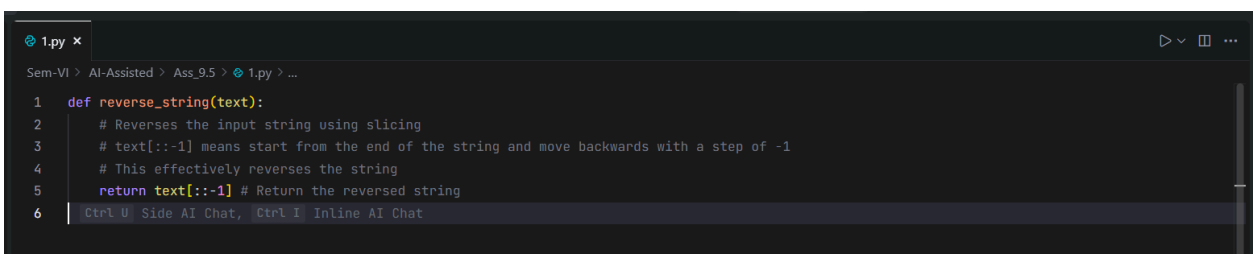
   Task:

   1. Write documentation in:
      a. Docstring
      b. Inline comments
      c. Google-style documentation
   2. Compare the three documentation styles.
   3. Recommend the most suitable style for a utility-based string library.

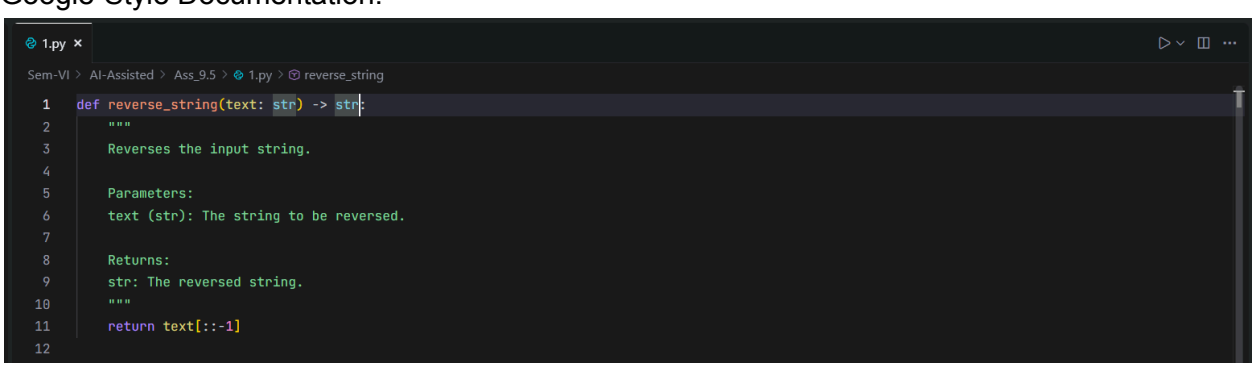
Docstring:



Inline Comments:



Google-Style Documentation:



Comparison:

Docstring Justification:

Provides a clear summary of the function's purpose, parameters, and return value in one structured block.

Inline Comments Justification:

Explain how the slicing logic works directly at the implementation level.

Google-Style Documentation Justification:

Offers a standardized and structured format that improves readability and tool integration.

Recommendation:

Google style is best for utility libraries because it supports scalability and automatic documentation generation.

2. Password Strength Checker

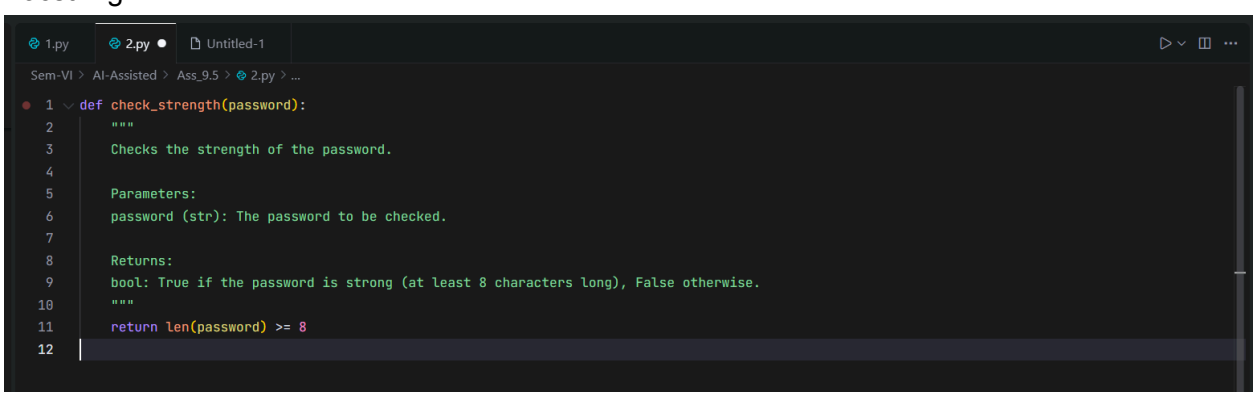
Consider the function:

```
def check_strength(password):
return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

Docstring:



Inline Comments:

Google-Style Documentation:

```python
def check_strength(password: str) -> bool:
    """
    Checks the strength of the password.

    Parameters:
    password (str): The password to be checked.

    Returns:
    bool: True if the password is strong (at least 8 characters long), False otherwise.
    """
    return len(password) >= 8
```

Comparison:

Docstring Justification:

Clearly defines the validation rule and expected return type for basic understanding.

Inline Comments Justification:

Highlights the specific security rule applied within the function logic.

Google-Style Documentation Justification:

Clearly documents input expectations and validation behavior in a structured format.

Recommendation:

Google style is most appropriate for security-related code because it explicitly defines validation rules and expected outputs.

3. Math Utilities Module

Task:

1. Create a module math_utils.py with functions:
    a. square(n)
    b. cube(n)
    c. factorial(n)
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

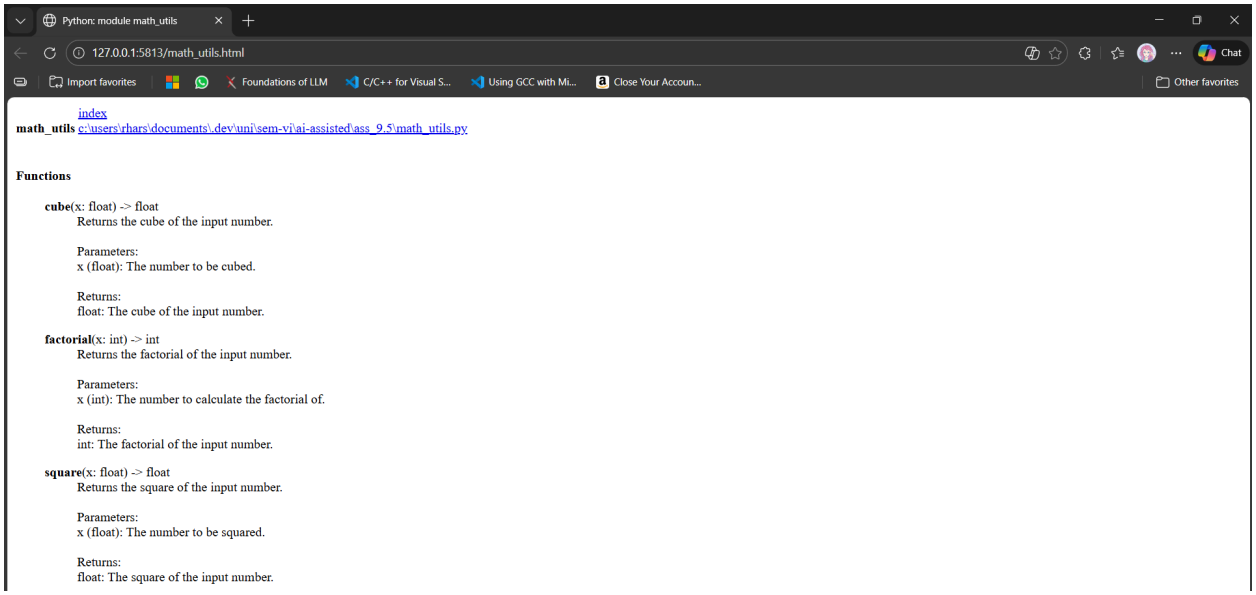
math_utils.py module with functions:

```python
def square(x: float) -> float:
    """
    Returns the square of the input number.

    Parameters:
    x (float): The number to be squared.

    Returns:
    float: The square of the input number.
    """
    return x ** 2

def cube(x: float) -> float:
    """
    Returns the cube of the input number.

    Parameters:
    x (float): The number to be cubed.

    Returns:
    float: The cube of the input number.
    """
    return x ** 3

def factorial(x: int) -> int:
    """
    Returns the factorial of the input number.

    Parameters:
    x (int): The number to calculate the factorial of.

    Returns:
    int: The factorial of the input number.
    """
    if x == 0:
        return 1
    else:
        return x * factorial(x - 1)
```

Exporting documentation as an HTML File.

```
PS C:\Users\rhars\Documents\.Dev\Uni> python -m pydoc -w C:\Users\rhars\Documents\.Dev\Uni\Sem-VI\AI-Assisted\Ass_9.5\math_utils.py
wrote math_utils.html
PS C:\Users\rhars\Documents\.Dev\Uni>
```

Viewing HTML file:



4. Attendance Management Module
   Task:
   1. Create a module attendance.py with functions:
      a. mark_present(student)
      b. mark_absent(student)
      c. get_attendance(student)
   2. Add proper docstrings.
   3. Generate and view documentation in terminal and browse

attendance.py module with functions with proper docstring:

1.py × 2.py math_utils.py attendance.py × Untitled-1

Sem-VI > AI-Assisted > Ass_9.5 > attendance.py > get_attendance

```python
def mark_present(student: str) -> None:
    """
    Marks the student as present.

    Parameters:
    student (str): The name of the student to mark as present.
    """
    print(f"{student} is marked as present.")

def mark_absent(student: str) -> None:
    """
    Marks the student as absent.

    Parameters:
    student (str): The name of the student to mark as absent.
    """
    print(f"{student} is marked as absent.")

def get_attendance(students: list) -> dict:
    """
    Returns the attendance record for a list of students.

    Parameters:
    students (list): A list of student names.

    Returns:
    dict: A dictionary containing the attendance record for each student.
    """
    attendance = {}
    for student in students:
        attendance[student] = "present"
    return attendance
```

Terminal:

```
Problems   Output   Debug Console   Terminal                                   powershell

Help on module attendance:

NAME
    attendance

FUNCTIONS
    get_attendance(students: list) -> dict
        Returns the attendance record for a list of students.

        Parameters:
        students (list): A list of student names.

        Returns:
        dict: A dictionary containing the attendance record for each student.

    mark_absent(student: str) -> None
        Marks the student as absent.

        Parameters:
        student (str): The name of the student to mark as absent.

    mark_present(student: str) -> None
        Marks the student as present.

        Parameters:
        student (str): The name of the student to mark as present.

FILE
    c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_9.5\attendance.py


PS C:\Users\rhars\Documents\.Dev\Uni>
```

5. File Handling Function
   Consider the function:

```python
def read_file(filename):
with open(filename, 'r') as f:
return f.read()
```

Task:

- Write documentation using all three formats.
- Identify which style best explains exception handling.
- Justify your recommendation.

Docstring:

```
1.py   2.py   math_utils.py   attendance.py   5.py ●   notes.txt ●   Untitled-1
Sem-VI > AI-Assisted > Ass_9.5 > 5.py > read_file > filename
1  def read_file(filename):
2      """
3      Reads the contents of a file and returns it as a string.
4
5      Parameters:
6      filename (str): The name of the file to read.
7
8      Returns:
9      str: The contents of the file as a string.
10     """
11     with open(filename, 'r') as f:
12         # Read the contents of the file
13         contents = f.read()
14         return contents
```

Inline Comments:

```python
def read_file(filename):
    # Open the file in read mode
    # with statement ensures that the file is properly closed after reading
    # even if an exception is raised
    # Read the contents of the file
    with open(filename, 'r') as f:
        contents = f.read()
        # Return the contents of the file
        return contents
```

Google-Style Documentation;

```python
def read_file(filename: str) -> str:
    """
    Reads the contents of a file and returns it as a string.

    Parameters:
    filename (str): The name of the file to read.

    Returns:
    str: The contents of the file as a string.
    """
    with open(filename, 'r') as f:
        # Read the contents of the file
        contents = f.read()
        return contents
```

Comparison:

Docstring Justification:

Clearly documents function behavior and possible exceptions in a structured manner.

Inline Comments Justification:

Explain file operations step-by-step but do not formally document errors.

Google-Style Documentation Justification:

Explicitly separates the Raises section, making exception handling clear and professional.

Recommendation:

Google style is best because it formally documents exceptions in a dedicated section, improving clarity and maintainability.